

Sentencias de control

Índice

- Sentencias de control
- Sentencias condicionales
- Sentencias repetitivas

Sentencias de control

Sentencias de control

Teorema de Böhm y Jacopini

La programación estructurada se basa en programas PROPIOS

- Tienen un solo punto de entrada y un solo punto de salida
- Toda acción del algoritmo es accesible pues existe al menos un camino que va desde el inicio hasta el fin del algoritmo pasando a través de dicha acción
- No tienen bucles infinitos

TEOREMA: todo algoritmo (y por lo tanto, cualquier programa) se puede construir con sólo 3 componentes estructurales

- Secuencia
- Selección
- Repetición

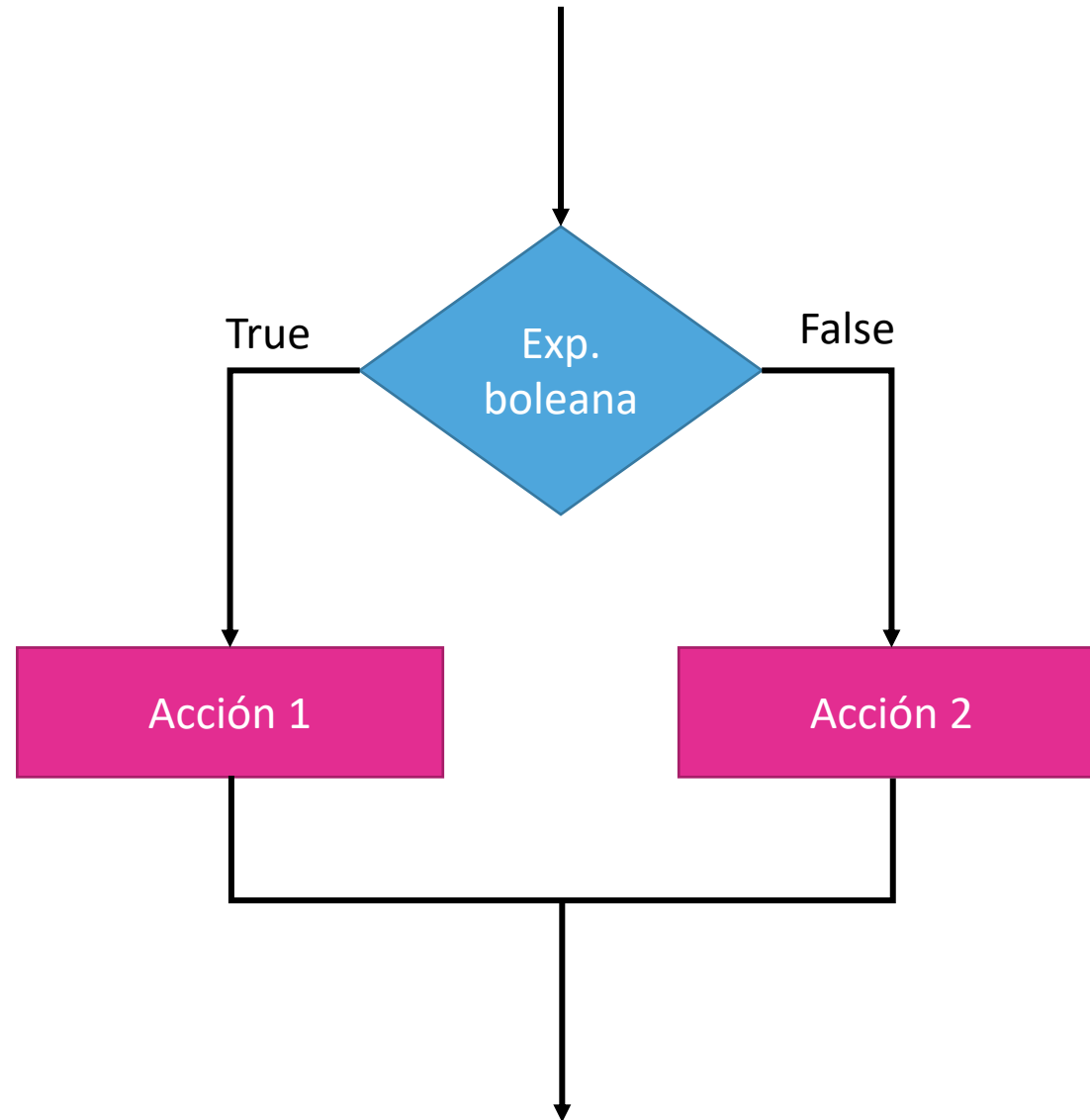
Sentencias de control

3 Componentes estructurales

- Secuencia: grupo de acciones que se ejecutan en serie
- Selección: permite elegir un camino entre varios según el valor de una condición, de forma que si es cierto se realiza una acción (o grupo de acciones) y si es falso se realiza otra acción (o grupo de acciones)
- Repetición: permite volver a ejecutar un conjunto de acciones una cierta cantidad de veces

Sentencias condicionales

Sentencia condicional



IF-ELSE

El formato para la sentencia condicional if-else es el siguiente:

```
if expresión_booleana:  
    Sentencias
```

```
else:  
    Sentencias
```

Ejemplo:

```
>>> if edad < 18:  
...     print ('Aún no has alcanzado la edad adulta')  
... else:  
...     print('Legalmente eres considerado un adulto')
```

IF-ELSE

```
>>> edad = 17
>>> if edad < 18:
...     print ('Aún no has alcanzado la edad adulta')
... else:
...     print('Legalmente eres considerado un adulto')
...
Aún no has alcanzado la edad adulta
```

```
>>> if edad < 18:
...     print ('Aún no has alcanzado la edad adulta')
... else:
...     print('Legalmente eres considerado un adulto')
...
Legalmente eres considerado un adulto
```

IF-ELSE

IF-ELSE es una sentencia y como tal puede estar dentro de otro IF-ELSE

```
if expresión_booleana:
    if expresión_booleana:
        Sentencias
    else:
        Sentencias
else:
    Sentencias
```

Ejemplo:

```
>>> if edad >= 18:
...     if edad >= 67:
...         print('Deberías disfrutar de tu jubilación')
...     else:
...         print('Eres un adulto que aún no puede jubilarse')
... else:
...     print('Aún no eres un adulto')
```

IF-ELSE

```
>>> edad = 17
>>> if edad >= 18:
...     if edad >= 67:
...         print('Deberías disfrutar de tu jubilación')
...     else:
...         print('Eres un adulto que aún no puede jubilarse')
... else:
...     print('Aún no eres un adulto')
Aún no eres un adulto
```

```
>>> edad = 19
>>> if edad >= 18:
...     if edad >= 67:
...         print('Deberías disfrutar de tu jubilación')
...     else:
...         print('Eres un adulto que aún no puede jubilarse')
... else:
...     print('Aún no eres un adulto')
Eres un adulto que aún no puede jubilarse
```

```
>>> edad = 71
>>> if edad >= 18:
...     if edad >= 67:
...         print('Deberías disfrutar de tu jubilación')
...     else:
...         print('Eres un adulto que aún no puede jubilarse')
... else:
...     print('Aún no eres un adulto')
Deberías disfrutar de tu jubilación
```

IF-ELSE

Aunque se pueden anidar varias sentencias if-else, existe una manera simplificada de agruparlos

```
if expresión_booleana:  
    Sentencias  
elif expresión_booleana:  
    Sentencias  
else:  
    Sentencias
```

Ejemplo:

```
>>> if edad >= 67:  
...     print('Deberías estar jubilado')  
... elif edad >= 18:  
...     print('Eres un adulto')  
... else:  
...     print('Aún no has alcanzado la edad adulta')
```

IF-ELSE

La sentencia IF-ELSE no tiene porque tener la parte else

if expresión_booleana:

Sentencias

Ejemplo:

```
>>> v = -3
```

```
>>> if v < 0:
```

```
...     v *= -1
```

```
...
```

```
>>> print('Valor absoluto: {}'.format(v))
```

```
Valor absoluto: 3
```

```
>>> v = 3
```

```
>>> if v < 0:
```

```
...     v *= -1
```

```
...
```

```
>>> print('Valor absoluto: {}'.format(v))
```

```
Valor absoluto: 3
```

Asignación condicional

Python tiene un operador ternario para la condición que se usa para asignar valores. El formato es el siguiente:

`a if condición else b`

Ejemplo

```
>>> a = 3
>>> b = 2
>>> maximo = a if a > b else b
>>> máximo
3
```

```
>>> a = 1
>>> b = 2
>>> maximo = a if a > b else b
>>> maximo
2
```

Sentencias repetitivas

Sentencias repetitivas

- Permiten programar la repetición de una o más sentencias (cuerpo del bucle) mediante la construcción denominada ciclo o bucle.
- En C, cada bucle tiene una expresión de control. El número de veces que se repite el cuerpo del bucle está determinado por la sección de control del bucle. Si la expresión es verdadera el bucle continúa ejecutándose.
- Cada vez que se ejecuta el cuerpo del bucle (una iteración del bucle), se evalúa la expresión de control.
- Debe existir una condición del final del bucle.

Sentencias repetitivas

- Dos tipos de bucles: el ciclo condicional y el ciclo sobre secuencias

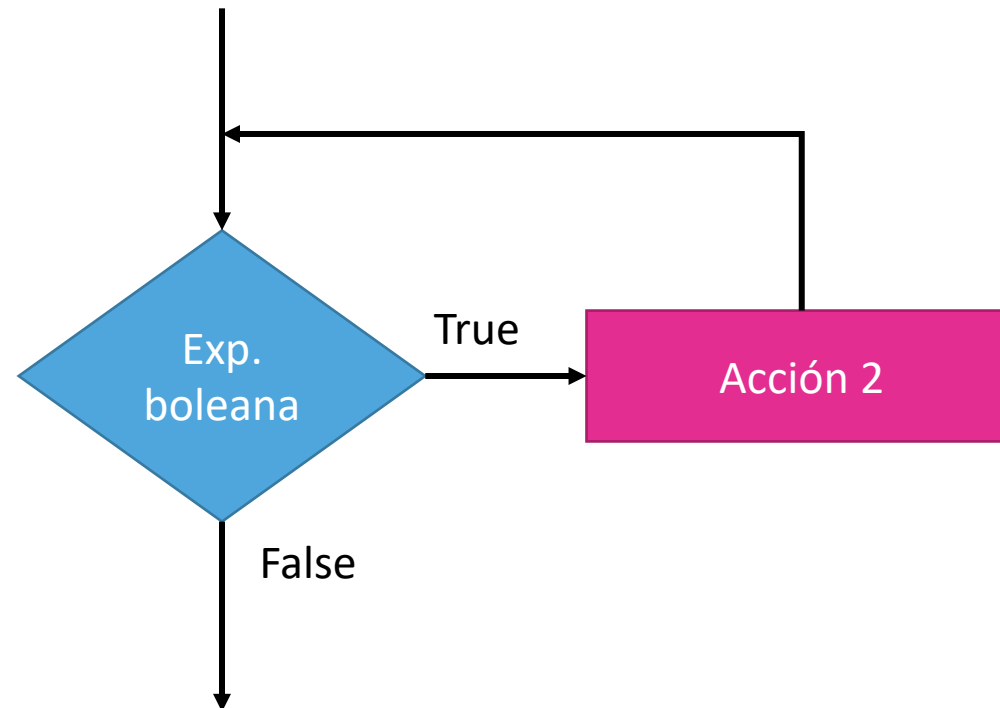
- Ciclo condicional

```
while condición:  
    Sentencias
```

- Ciclo secuencial

```
for elemento in secuencia:  
    sentencias
```

Bucle while



Bucle while

El bucle while permite de una manera sencilla configurar un grupo de sentencias que se ejecutarán de manera repetida mientras se cumpla una condición.

Ejemplo:

```
>>> while i < n:    # expresión de control
...     i *= 2      # cuerpo del bucle
```

Bucle while

Supongamos que n vale 5 e i 1 en el bucle anterior

```
>>> n = 5
>>> i = 1
>>> while i < n:
...     i *= 2
```

i=1	i es 1
¿es i < n?	Sí, ejecuta el cuerpo del bucle
i *= 2	i es ahora 2
¿es i < n?	Sí, ejecuta el cuerpo del bucle
i *= 2	i es ahora 4
¿es i < n?	Sí, ejecuta el cuerpo del bucle
i *= 2	i es ahora 8
¿es i < n?	No; se sale del bucle

Bucle while

```
>>> n = 5
>>> i = 1
>>> while i < n:    # expresión de control
...     i *= 2      # cuerpo del bucle
...     print(i)    # cuerpo del bucle
...
2
4
8
```

Bucle for

En la mayoría de lenguajes, el bucle for se usa como un bucle de “conteo” para hacer una tarea durante un numero de repeticiones determinado. Sin embargo, en Python el comportamiento del bucle for es diferente y sirve para recorrer los elementos de una secuencia o lista.

```
>>> for i in range(5):  
...     print(i)  
...  
0  
1  
2  
3  
4
```

range(n) es una función de Python que devuelve una lista de números entre 0 y n-1. El uso de esta función y el funcionamiento de listas se verán apartados posteriores

Sentencias especiales en los bucles

En Python, como en muchos lenguajes, existen las sentencias `break` y `continue` para modificar el comportamiento del bucle.

- `break`: El bucle finaliza y se continua ejecutando las sentencias fuera del bucle
- `continue`: Se salta la iteración del bucle y procede a ejecutar la siguiente iteración

Hay que prestar mucha atención al uso de estas sentencias ya que un uso no adecuado puede producir un comportamiento no deseado.

Sentencias especiales en los bucles

```
>>> for i in range(3):  
...     if i == 1:  
...         continue  
...     print(i)  
0  
2
```

```
>>> i = 0  
>>> while i < 3:  
...     i += 1  
...     if i == 2:  
...         continue  
...     print(i)  
1  
3
```

```
>>> for i in range(3):  
...     if i == 1:  
...         break  
...     print(i)  
0
```

```
>>> i = 0  
>>> while i < 3:  
...     i += 1  
...     if i == 2:  
...         break  
...     print(i)  
1
```

Diseño de bucles

Seleccionar el tipo de bucle

- El bucle for sirve para iterar sobre los elementos de una lista y es necesario tener una lista para usarlos. Si se quiere iterar un número determinado de veces se puede usar este tipo de bucle usando la función range(n)
- El bucle while se suele utilizar cuando no se conoce el número de repeticiones y es necesario evaluar una condición antes de entrar en el bucle

Consejos

- Inicializar la variable de control en el bucle while
- Construir correctamente la condición de salida en los bucles while
- Alterar en algún momento la variable de control en el cuerpo en los bucles while
- Comprobar que se alcanza la condición de terminación

Errores en los bucles

Inicialización

While condición:

...

Alteración de la condición

- No inicializar la variable de control -> Resultado indeterminado
- No alteración de la variable de control en el cuerpo -> bucle infinito
- No se alcanza la condición de terminación -> bucle infinito
- Mala sintaxis -> resultados inesperados
- Condición mal construida